Project IQ - Features, Achievements & Integrations

This document provides a comprehensive overview of the capabilities, technical achievements, and integrations of the Project IQ Chatbot application.

1. AI & NLU Capabilities (Rasa Core + NLU)

The core of the chatbot is built on Rasa 3.x, utilizing advanced machine learning policies for robust dialogue management.

Intents & Dialogue Management

- Ticket Lifecycle Management:
  - create_ticket: Guided flows to report issues.
  - check_status: Real-time status lookup by Ticket ID.
  - list_tickets: View active or closed tickets for the current user.
  - update_ticket: Add new information to existing tickets.
  - verify_resolution: Confirm if a solution worked or reopen the ticket.
- Knowledge Base Integration:
  - ask_knowledge: Search the KB for solutions.
  - kb_not_helpful: Fallback mechanism to escalate KB queries to ticket creation.
- User Interaction:
  - greet, goodbye: Natural conversation flow.
  - help: Context-aware assistance.
  - clear: Resets the conversation session.
- Feedback Mechanism:
  - feedback_positive / feedback_negative: Sentiment tracking.
  - save_satisfaction_rating: 1-5 star rating system stored for analytics.

Smart Form Handling

• Interactive Slot Filling: Uses ticket_form to collect required data:

  • Title/Summary: ticket_title

  • Description: ticket_content

  • Categorization: issue_type (Hardware, Software, Network).

  • Priority: urgency (Low, Medium, High).

• Context Awareness: Can extract context from previous messages to pre-fill slots.

GuardRails & Safety

• Edge Case Detection:

  • Fallback Handling: Uses FallbackClassifier and UnexpecTEDIntentPolicy to catch low-confidence inputs.

  • Scope Management: Gracefully declines out-of-scope requests (e.g., "Make me a sandwich") with utter_out_of_scope.

  • Error Recovery: Automated error handling in actions ensures the bot apologizes and asks for clarification instead of crashing.

• Multi-Intent Handling (ActionHandleMultiIntent):

  • Intelligently parses complex user requests containing multiple actions (e.g., "Create a ticket and check my status").

  • Queues actions and guides the user through them sequentially to prevent confusion.

• KB Deflection (ActionSearchKbBeforeTicket):

  • Proactively searches the Knowledge Base before creating a ticket.

  • Reduces support load by suggesting solutions first.

• Auth Guardrails:

  • Strict check_user_login validation on all protected actions.

  • Prevents unauthorized access to ticket data.

Advanced AI Policies

• TEDPolicy (Transformer Embedding Dialogue): Handles complex, non-linear conversation paths using machine learning.

• RulePolicy: Enforces strict business logic (e.g., "Always ask for confirmation before submitting").

• UnexpecTEDIntentPolicy: Gracefully handles unexpected user inputs by guiding them back to supported flows.

Rich Responses

• Interactive Elements: Buttons for quick choices.

• Data Visualization:

  • action_show_chart: Renders dynamic charts.

  • action_show_ticket_graph: Visualizes ticket statistics.

• --

2. Backend Integrations

Technical integration points that power the application's logic.

GLPI (IT Service Management)

The bot is deeply integrated with GLPI via its REST API.

• CRUD Operations: Full Create, Read, Update access to Tickets.

• Authentication:

  • action_glpi_login: Validates user credentials against GLPI.

  • Profile Checks: Enforces permissions (e.g., restricting Admin features to "Super-Admin").

• Multi-Entity Support (Department Context):

  • Context Switching: Dynamically switches GLPI Entities (e.g., "IT" vs "Accounting") based on app_id.

  • Data Isolation: Ensures users only see tickets relevant to the current department context.

• Knowledge Base Sync:

  • kb_cache_manager.py: Background service that caches KB articles for millisecond-latency searches.

Database (PostgreSQL)

Replaces default SQLite for production-grade reliability.

• Custom Tracker Store:

  • Persists conversation history in conversations and messages tables.

  • Allows long-term storage and auditing of chat logs.

- Analytics Storage:
  - feedback table: Stores user ratings and comments.
  - actions table: Logs bot performance and action success rates.

Security (Infisical)

Enterprise-grade secret management.

- Secret Management: Uses Infisical to manage sensitive Environment Variables.
  - Protects GLPI_APP_TOKEN, DB_PASSWORD, and API Keys.
- Environment Isolation: Supports distinct secrets for dev and prod environments.

- --

3. Admin & Operations Suite

A comprehensive administration microservice (Port 8181) built with Flask + Vanilla JS.

1. Analytics & Reporting (admin-analytics.js)

- Real-time Dashboard: Visual graphs for Conversation Counts, Intent Distribution, and Sentiment Analysis.
- Resolution Tracking: Monitors the "Ticket Resolution Rate" based on positive/negative user feedback.
- Deep Filtering: Slice data by Date Range (24h, 7d, 30d, Custom), App ID (IT, HR), or Ticket Status.
- Chat History Filters: Advanced filtering capabilities for conversation history and logs:
  - Filter by Date Range (specific dates, last 24 hours, 7 days, 30 days).
  - Filter by User ID or App ID.
  - Filter by Intent Type (create_ticket, check_status, etc.).
  - Search by Message Content (full-text search).
  - Export filtered conversations for auditing and compliance.

2. AI Studio & Debugging (admin-studio.js)

- Interactive Chat Tester:
  - Test bot responses in real-time without deploying.

    - NLU Analyzer: View raw Intent Confidence and Entity Extraction results for every message.
    - Response Preview: See exactly which text or custom payload the bot selected.
- Flow Visualizer: Renders dialogue paths (stories/rules) as Mermaid.js flowcharts.
- Response Manager: Catalog of all bot responses and variations.

3. System Observability (admin-observability.js)

- Health Monitor: Live status checks for Rasa Core, Action Server, and PostgreSQL Database.
- Resource Tracking: Real-time gauges for CPU, Memory, and Disk usage via process_monitor.py.
- Log Viewer: Built-in GUI to stream, search, and filter server logs (INFO/ERROR) via log_parser.py.
- Database Stats: Metrics on table sizes and transaction rates.

4. Knowledge Base Management (admin-knowledge-base.js)

- Cache Controls: View status of the In-Memory Article Cache and execution of Vector Embeddings.
- Manual Refresh: Force-sync KB articles from GLPI immediately.
- Search Tester: Test the "Hybrid Matching" algorithm (Semantic + Keyword) directly in the Admin UI to tune results.

5. Conversation Inspector (admin-inspector.js)

- Turn-by-Turn Replay: detailed view of any past conversation.
- Slot Inspection: View the precise state of all memory slots at any point.
- Flow Tracing: Identify exactly why a fallback occurred or a rule was triggered.

6. Configuration & Theming

- Theme Builder (admin-themes.js):
    - No-code editor to customize the Chat Widget's appearance (Colors, Fonts, Radius).
    - Dark Mode Generator: One-click creation of dark-themed presets.
- Config Manager (admin-config.js): Hot-swap system settings (GLPI URLs, Keys) without server restarts.

- --

4. Frontend & User Experience

A modern, responsive client interface designed for seamless interaction.

Observability & Analytics

• Real-time Process Monitoring:

  • Tracks CPU, Memory, and Disk usage via actions/process_monitor.py.

  • Monitors status of Rasa Core, Action Server, and Database services.

• Log Analysis & Filtering:

  • Built-in log parser (actions/log_parser.py) for viewing and filtering system logs (INFO, ERROR, DEBUG) directly from the Admin UI.

• Conversation Inspector:

  • Turn-by-Turn Debugging: View full conversation history with intent confidence scores and active slots.

  • Flow Visualization: Graphical representation of dialogue flows (inspector-flows-grid).

  • NLU Tester: Live testing interface for intent classification and entity extraction.

• Business Analytics:

  • Tracks "Resolution Rate", "Top Intents", and "User Feedback" (Sentiment Analysis).

Knowledge Base Engine

• Hybrid Search Technology:

  • Combines Semantic Search (using sentence-transformers) with Keyword Matching for high-accuracy results (actions/kb_cache_manager.py).

  • Scoring Algorithm: Weighted ranking (60% Semantic / 40% Keyword) to surface the best answers.

• Performance Optimization:

  • In-memory caching with embeddings storage for millisecond-latency responses.

  • Auto-Refresh: Configurable interval to sync with GLPI KB updates.

• Sources:

  • Primary: Live GLPI Knowledge Base (API).

  • Fallback: Local JSON files for offline redundancy.

Backend Infrastructure

• Database Resilience:

  • PostgreSQL connection pooling (actions/conversation_db.py) for handling high concurrency.

  • Auto-Migration: Scripts to migrate data from SQLite to PostgreSQL (scripts/migrate_to_postgres.py).

• Security:

  • Secret Management: Integration with Infisical for secure credential injection.

  • Session Management: Secure token handling and automated cleanup (ActionClearSession).

• Automation Utilities:

  • Presentation Generator: Automated PowerPoint creation for reporting (scripts/create_presentation.py).

  • Theme Generator: Scriptable theme creation (scripts/create_default_themes.py).

Smart Chat Widget

• Tech Stack: Vite + TailwindCSS + Vanilla JS.

• Architecture:

  • Compiles to a single embeddable file: glpi-chat-widget.js.

  • Can be dropped into any HTML page or IT Portal.

• Theming Engine & Custom Base:

  • Pre-built Themes: Includes 8 professional presets defined in scripts/create_default_themes.py (e.g., Ocean Blue, Forest Green).

  • Customization: Full control over Colors, Fonts, Border Radius, and Icons via Admin UI.

  • Live Preview: Real-time visual editor for theme adjustments.

• Advanced UI:

  • File Uploads: Drag-and-drop support (action_upload_file).

  • Rich Media: Renders images, links, markdown, and charts/graphs.

  • Voice Support: (Experimental) Text-to-Speech and Speech-to-Text capabilities.

Specialized Entry Points

• it.html: specialized landing page for IT Support.

• acc.html: Landing page for Accounts/HR support (extensible).

Privacy & Compliance

• Privacy Notice:
  • Prominent privacy disclosure displayed on chat widget initialization.
  • Informs users about data collection, storage, and usage practices.
  • Links to detailed privacy policy documentation.
  • Includes opt-in consent for analytics and feedback collection.
• Cookies Policy:
  • Clear cookie consent banner on first visit.
  • Categorizes cookies (Essential, Analytics, Preference).
  • Users can manage cookie preferences and opt-out of non-essential tracking.
  • Persistent consent tracking stored in browser local storage.
  • Compliant with GDPR, CCPA, and other privacy regulations.

• --

5. Technical Architecture & Achievements

Microservices Architecture

The system is decoupled for scalability and maintenance:

1. Rasa Core (5005): Handles NLU and Dialogue.
2. Action Server (5055): Executes Python business logic.
3. Admin API (8181): Manages configuration and analytics.
4. Nginx: Reverse proxy unified entry point (Ports 80/443).

Robustness & Reliability

• Threading & Caching:
  • kb_cache_manager.py uses background threads to keep data fresh without blocking user requests.
• Health Checks:
  • All services expose /health endpoints for monitoring.
• Dockerized Deployment:
  • Full docker-compose support with network isolation.

- Resilient Database Connections:
  - conversation_db.py implements a robust connection pool with auto-reconnection logic.